Community Dreams FOUNDATION

CDF HACKATHON / ORIENTATION / AGENTIC QA

Agentic QA Hackathon

Autonomous specification-driven QA testing

MISSION
Verify the PRD

TARGET
DemoBank

FORMAT
Code-first

The mission

THE CORE QUESTION

Does the application behave the way the specification says it should?

Your job

Build an agent that reaches defensible requirement-level verdicts.

From input to evidence

1. **Inputs**
   Target URL, optional repository and written PRD
2. **Agent**
   Interpret, explore, execute, recover and verify
3. **Output**
   Passed, failed and unverified results with evidence

The challenge rewards verification quality, not a long list of automated clicks.

Why this problem matters

1 **Requirements**

Product teams describe expected behavior in prose.

2 **Implementation**

A live application contains correct flows and hidden divergence.

3 **Verification**

Manual translation is slow and brittle after benign UI changes.

4 **Evidence**

Teams need a repeatable explanation of what passed or failed.

The gap to close

Your agent translates requirements into assertions, chooses useful scenarios and supports every verdict with observable facts.

The autonomous QA loop

1	**INTERPRET**	Convert PRD statements into structured, testable assertions
2	**PLAN**	Choose normal, negative and boundary scenarios
3	**EXECUTE**	Navigate the UI and observe the system
4	**VERIFY**	Compare expected and observed outcomes
5	**REPORT**	Publish verdicts, evidence, cost and gaps

A resilient agent can revise its plan, retry safely and explain what it could not verify.

Meet the target: DemoBank

BEHAVIORAL CONTRACT

PRD.md

Source of truth

The PRD defines expected user behavior. The running site may diverge from it.

Synthetic data only. No external bank connections.

A fictional banking dashboard

1	**Session**	Enter Demo, refresh, log out and protect private routes
2	**Accounts**	View balances and account-specific detail
3	**Transfers**	Move funds between owned accounts with validation
4	**History**	Filter combined and account-specific transactions
5	**Profile**	Show only masked synthetic customer information

What is in the starter package

1	**README.md**	Challenge, rubric, submission rules and safety boundaries
2	**test-website/**	React frontend, FastAPI backend and PostgreSQL data
3	**test-website/PRD.md**	Expected user-facing behavior your agent must verify
4	**test-website/LOCAL_SETUP.md**	Optional local startup and deterministic reset instructions
5	**agent/**	Workspace for the autonomous QA system

Treat the PRD as truth, the running app as evidence and target source as context rather than an answer key.

What your agent must deliver

01 REQUIREMENT INTERPRETATION

Structured assertions tied back to the PRD

02 AUTONOMOUS EXECUTION

Dynamic browser exploration with safe recovery

03 DISCREPANCY AND EVIDENCE

Expected versus observed behavior with proof

04 COVERAGE REPORTING

Passed, failed and unverified outcomes by requirement

Architecture is open

Browser automation, DOM analysis, multimodal grounding or a hybrid can work.

Coverage that earns confidence

Every important requirement deserves more than one happy-path check.

Normal flow	A valid transfer updates both balances and both account histories.
Negative flow	A rejected transfer leaves balances and history unchanged.
Edge case	Boundary values and filter dates follow the PRD exactly.

FOR EVERY SCENARIO

State the requirement

Capture the observed state

Check side effects before assigning a verdict.

A successful click does not prove that a requirement passed.

A useful QA verdict is auditable

1	**REQUIREMENT**	The exact behavior under review
2	**SCENARIO**	Preconditions and actions taken
3	**EXPECTED**	The outcome defined by the PRD
4	**OBSERVED**	Values, state and responses actually seen
5	**EVIDENCE**	Screenshot, DOM, network or console artifact
6	**VERDICT**	**Passed, failed or unverified with a reason**

PASS

Observed state matches the PRD.

FAIL

Evidence shows a meaningful mismatch.

UNVERIFIED

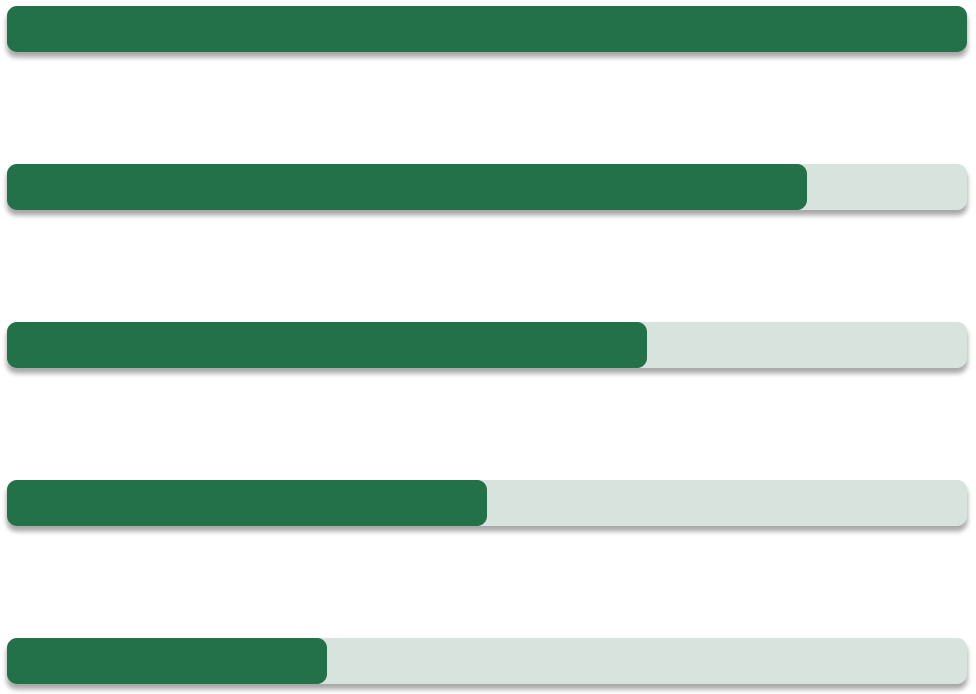
Explain why a reliable check was not possible.

What organizers provide

1	**HOSTED TARGET**	A DemoBank URL with deterministic, intentional product defects
2	**BEHAVIORAL SPECIFICATION**	A PRD covering access, accounts, transfers, history and profile
3	**STARTER REPOSITORY**	Target source, local setup instructions and an agent workspace
4	**SYNTHETIC TEST DATA**	One demo customer, two accounts and varied transactions

Guardrails **Use only synthetic data. Do not submit secrets, run denial-of-service tests or probe infrastructure outside the supplied app.**

How submissions will be judged



Accuracy and defect detection	**30**	Find real defects with few false positives
Evaluation and verification rigor	**25**	Cover positive, negative and edge cases
Cost of running the agent	**20**	Report practical API or token cost
Autonomous resilience	**15**	Recover from dynamic or unexpected states
Evidence and actionability	**10**	Give useful proof and reproduction steps

Accuracy plus verification rigor account for 55% of the score.

Your submission package

01 **PUBLIC CODE REPOSITORY** — Code

Agent source, setup scripts and a clear architecture README

02 **3–5 MINUTE WALKTHROUGH** — Video

Spec ingestion, live browser execution and generated artifacts

03 **QA AUDIT REPORT** — Report

Requirement-level results and reproduction evidence

04 **COST BREAKDOWN** — Cost

Estimated API or token cost for the demonstrated run

Make the demo reproducible.

Include environment requirements, run commands and known limitations.

Get started

1	**Read**	Map every PRD behavior to a requirement ID.
2	**Run**	Open the hosted app or use the local setup to learn the main flows.
3	**Baseline**	Execute one requirement end to end with captured evidence.
4	**Expand**	Add negative and boundary scenarios, then recovery behavior.
5	**Measure**	Track coverage, false positives, runtime and cost.

YOUR NEXT MOVE

Build toward a trustworthy answer to the specification.